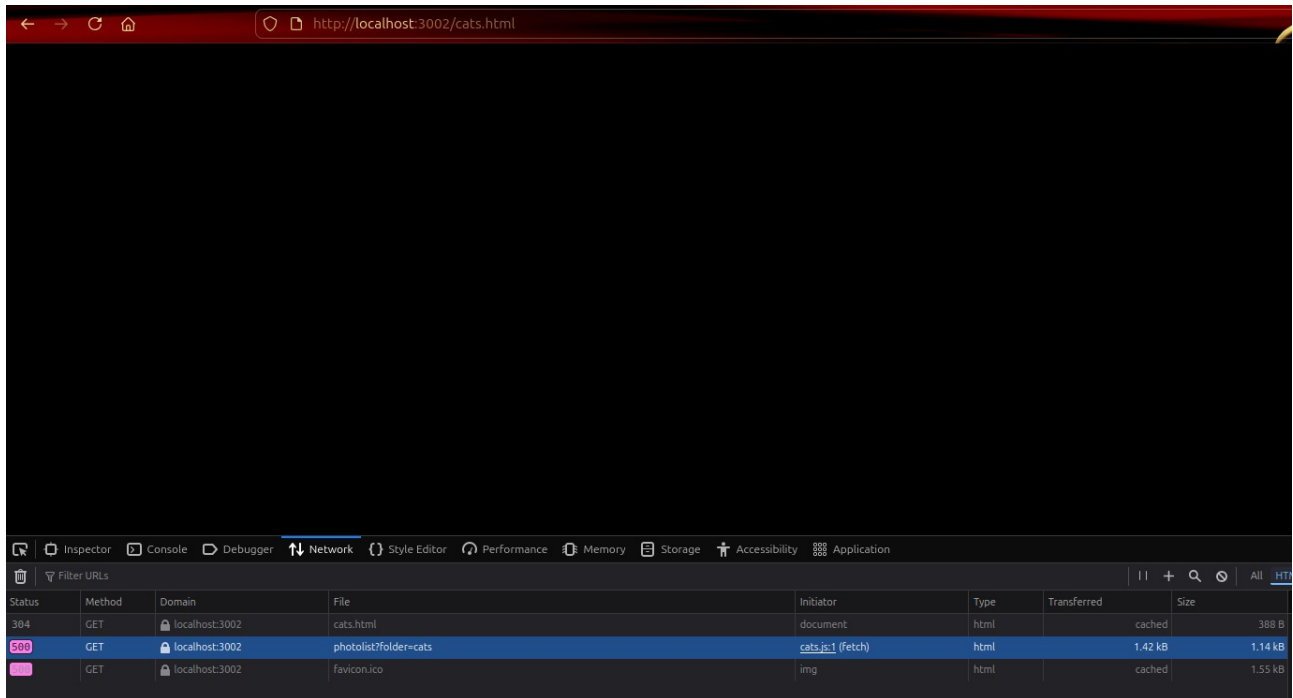


3. TRAVERSAL ATTACK

Preconditions

The workshop file was downloaded from the nodehill website, unzipped and located in a easily accessible folder then processed through the Visual Code Studio → Open folder → npm install → node index.js, just like other projects in the Web Security module.

After the setup however, I have experienced significant issues with processing the website content. The browser the <http://localhost:3002/> resolved to the website but the Dogs/ Cats compartments yielded black screen, no content. The Console rendered API (/api/photolist?folder=cats) with 500 error



the usual traversal paths I have tried:

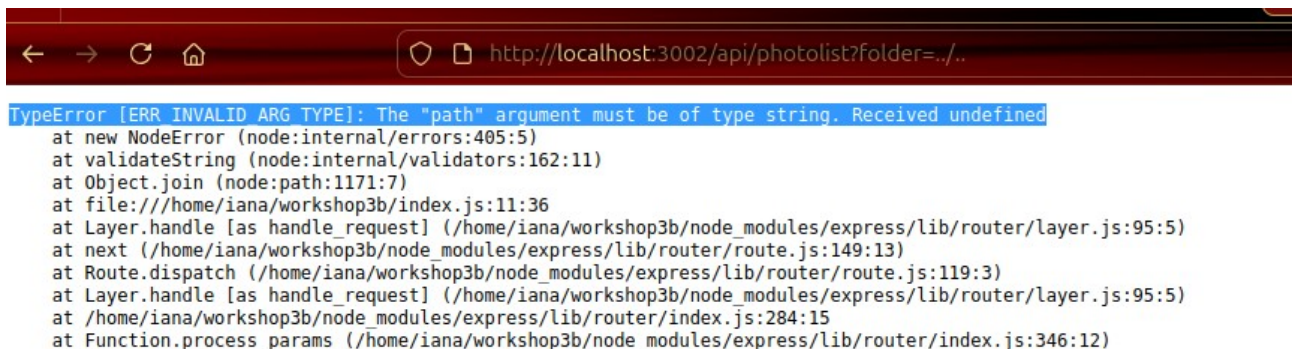
/api/photolist?folder=../index.js

/api/photolist?folder=../..

/api/photo?file=../package.json

/api/photolist?folder=home/iana/workshop3b/images/cats

and multiple variations of the above came down to *TypeError [ERR_INVALID_ARG_TYPE]: The "path" argument must be of type string. Received undefined* (as per screenshot)



This exposed an absolute path in my file system through stack trace → `/home/iana/workshop3b/index.js` and the directory structure on the server/my computer. It is also a point in case of CWE-209: Information Exposure Through an Error Message, though not exactly intentional. What this shows is a directory traversal vulnerability but doesn't yet constitute a traversal attack. Instead it reveals internal file paths setting up conditions for potential DOS attacks.

The issue of not being able to conduct successful traversal attack on my environment happens because the application crashes before the filepath can be accessed. In the source code `import.meta.dirname`, which represents the URL of the operative file (`file:///home/iana/workshop3b/index.js`) is returned as "undefined" every time Node tries to resolve `path.join`, no matter the traversal URL and syntax.

In the source code (`index.js`) I add the following line:

```
import {fileURLToPath } from 'url';  
const __dirname = path.dirname(fileURLToPath(import.meta.url));
```

which converts "`file:///...`" URL into the file path and registers it as `__dirname` as a destination folder. Additionally `import.meta.dirname` is now changed to `__dirname`.

The following changes don't increase the the vulnerability of the code in a meaningful way in addition to vulnerabilities already present in the original code. The changes are only made for the purpose of conducting traversal attack

New code after changes:

```
import { fileURLToPath } from 'url';  
const __dirname = path.dirname(fileURLToPath(import.meta.url));  
  
const port = 3002;  
const app = express();  
app.use(express.static('frontend'));  
  
app.get('/api/photolist', (req, res) => {  
  const list = fs.readdirSync(path.join(__dirname, 'images', req.query.folder));  
  res.json(list);  
});  
  
app.get('/api/photo', (req, res) => {  
  res.sendFile(path.join(__dirname, 'images', req.query.file));  
});  
  
app.get('*', ((_req, res) =>  
  res.sendFile(path.join(__dirname, 'frontend', 'index.html'))));  
  
app.listen(port, () => console.log('Listening on http://localhost:' + port));
```

With this code implementation I can now run traversal attacks.

Sequence of events

1. Upon code alternation and setting up the website on the Visual Code, the url <http://localhost:3002/cats.html> now shows pictures of cats and dogs. The api is there- Dev tools → Console → reload → API: **photolist?folder=** for folders and **photo?file=** for files

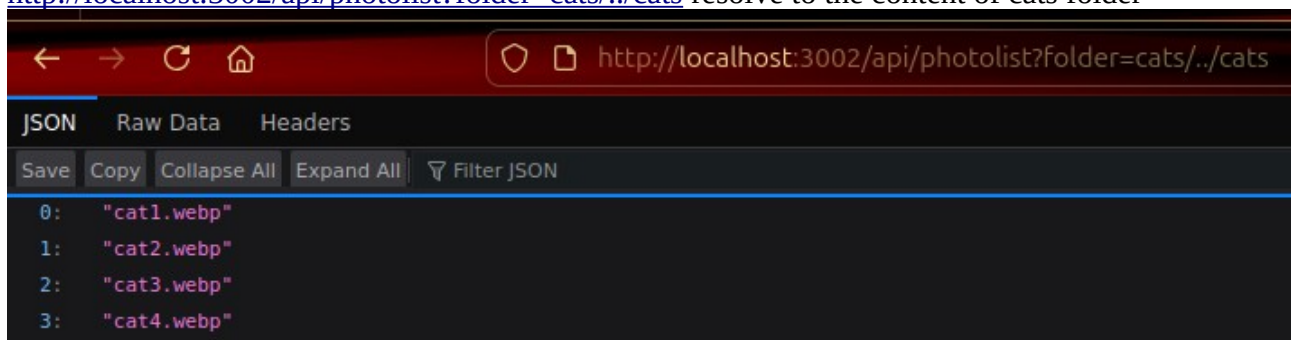
The api is photolist?folder=cats which will be used to gain access to the folder within the “images”

2. The API does not validate or restrict the folder parameter.

By supplying traversal sequences (. .) in the request, it is possible to move outside the intended directory. <http://localhost:3002/api/photolist?folder=cats/../../> allow access to the entire content of the /images folder and beyond

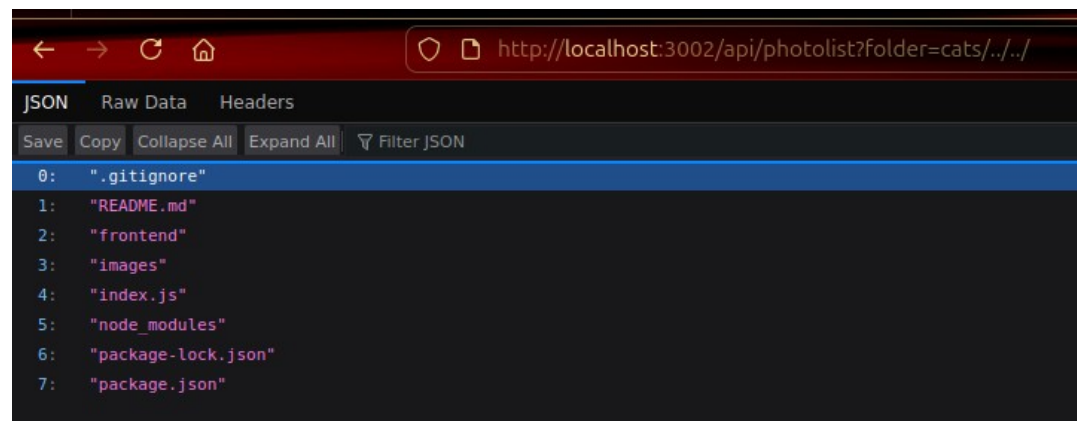
Examples (not exhaustive):

<http://localhost:3002/api/photolist?folder=cats/../../cats> resolve to the content of cats folder



Repeating this traversal allows access to directories sideways to the intended scope of access.

At this stage, the API exposes files that are not meant to be accessible through the web interface such as the content to the entire workshop file, including the source code.



The content of the source code can be further invoked though <http://localhost:3002/api/photo?file=..%2Findex.js> (screenshot below)

```
import fs from 'fs';
import path from 'path';
import express from 'express';

import { fileURLToPath } from 'url';
const __dirname = path.dirname(fileURLToPath(import.meta.url));

const port = 3002;
const app = express();
app.use(express.static('frontend'));

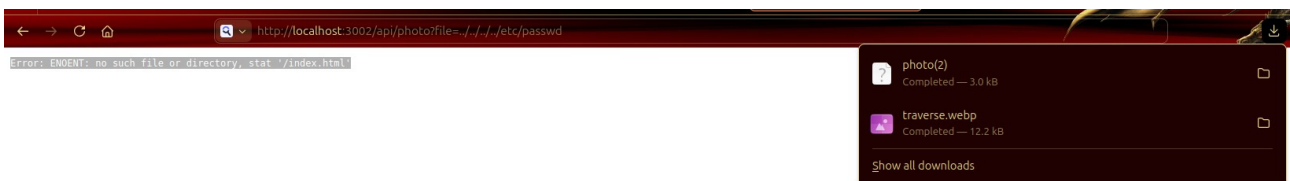
app.get('/api/photolist', (req, res) => {
  console.log('folder parameter:', req.query.folder);
  const list = fs.readdirSync(path.join(__dirname, 'images', req.query.folder));
  res.json(list);
});

app.get('/api/photo', (req, res) => {
  res.sendFile(path.join(__dirname, 'images', req.query.file));
});

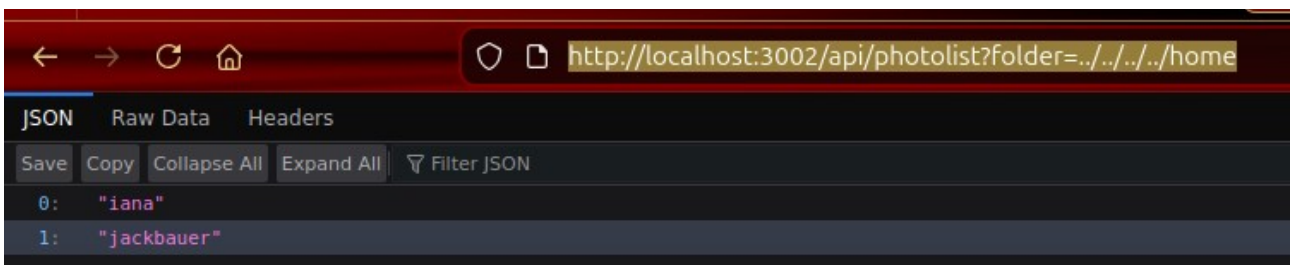
app.get('*', ((_req, res) =>
  res.sendFile(path.join(__dirname, 'frontend', 'index.html'))));

app.listen(port, () => console.log('Listening on http://localhost:' + port));
```

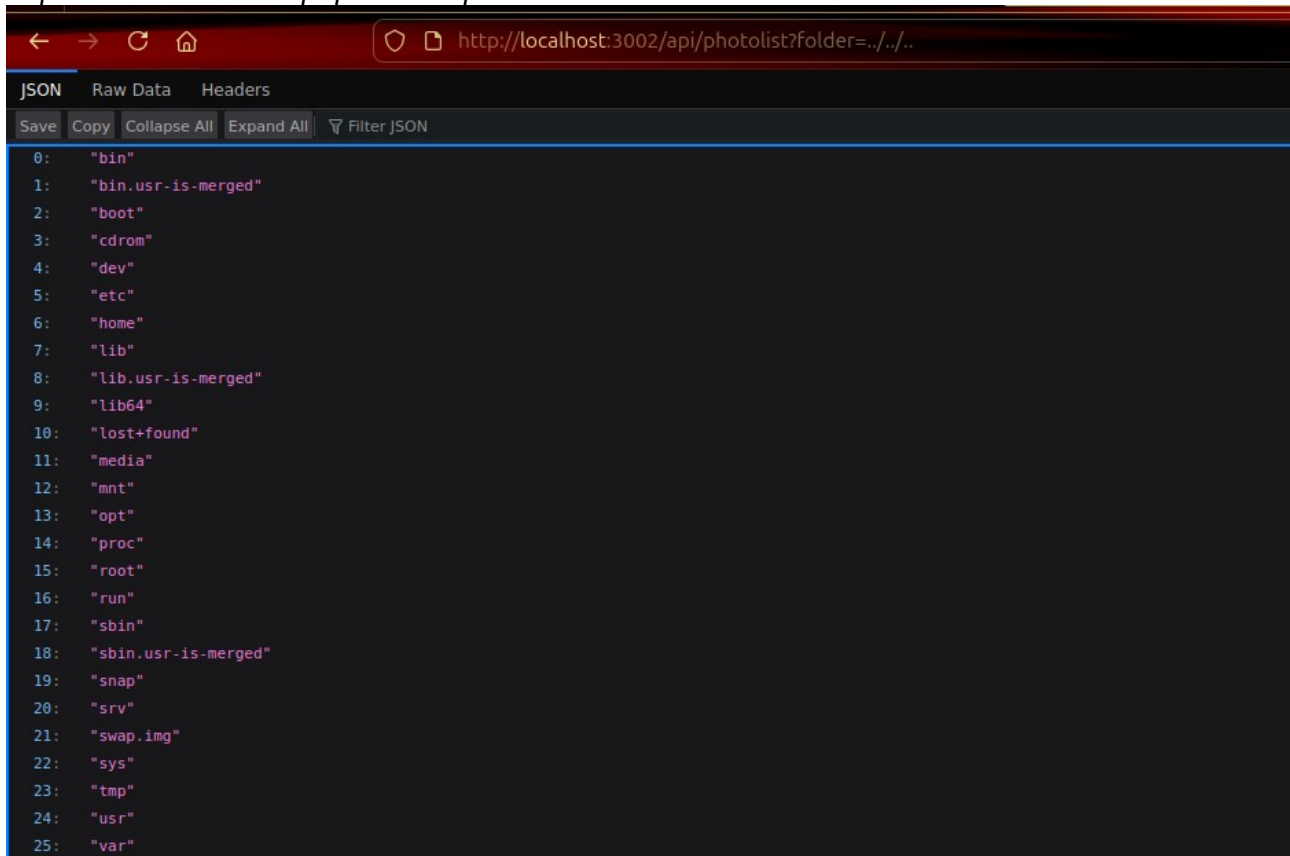
User accounts and details are automatically downloaded to /Downloads through <http://localhost:3002/api/photo?file=../../etc/passwd>



User accounts on the OS can be accessed through <http://localhost:3002/api/photolist?folder=../../home>



Last but not least the content of the / dir is displayed entirely through <http://localhost:3002/api/photolist?folder=../../../../../>



A series of attempts above show examples of how files across the entire OS can be accessed from the workshop folder to internal root documents. The above samples are not exhaustive but only to show examples of traverse attack in action.

The content can be also obtained and directly downloaded through curl command

The screenshot below shows `curl http://localhost:3002/api/photo?file=../../../../../index.js -o index.js` attempt to download index.js via traversal attempt and downloaded folder in /home/Desktop



Why did the attack work? What was it in the code that made it possible?

There are several attack vectors in the code that can be exploited

1. `fs.readdirSync(path.join(import.meta.dirname, 'images', req.query.folder));` and `res.sendFile(path.join(import.meta.dirname, 'images', req.query.file));`

`req.query.folder` and `req.query.file` is whatever the user types after `/api/photolist?folder=` and `/api/photo?file=` respectively.

These pieces of code are directly responsible for allowing direct access to the filesystem. Values taken from `req.query.folder` and `req.query.file` and are never restricted by removing `../` sequences, limiting filenames to a known list of image files, or verifying that the final resolved path still starts with the `/images` directory.. These values are then used to construct real filesystem paths on the server. Because `path.join()` merges path segments while indiscriminately trusting user input, by default it allows travelling well outside the `/images` directory. It means that the “`../`” input is happily resolved by the code to reconstruct filesystem paths on the server.

2. `app.get('/api/photo', (req, res) => {
 res.sendFile(path.join(import.meta.dirname, 'images', req.query.file)) });`

This code works similar to the one above but with one difference that the browser sends GET requests to `/api/photo` and the file content that's being retrieved can be sent to a user-controlled output

3. `fs.readdirSync(...)`

This piece of code reads the content of a dir: `fs.readdirSync('/home/iana/workshop3b/images')`. Followed by `fs.readdirSync(path.join(__dirname, 'images', req.query.folder))` it gives the user to power to chose which dir he wants to access. As explained in point 1, by listing `../` the intruder can trespass `/images` folder and wander around the system listing and reading files every which way. With this in mind, there is no need to know the OS or filesystem. One can just try traversal sequences (`../`) and `fs.readdirSync(...)` will show exactly what exists step by step by probing locations through trial and error.

Which category in the OWASP Top 10 belongs to the vulnerability?

The attack fits best **OWASP AD01:2025- Broken Access Control** because the nature of the code fails to enforce limits as to what a hacker can enter and download.

How did you secure the code? Show what you changed and explain why it works.

The simplest and quickest way of securing the code would be to get rid of the vulnerable points described in the point by attempting to define a folder of safe access and restrict traversal movement to the contents of that folder only.

This is shown by the proposed code fragment change:

```
app.get('/api/photolist', (req, res) => {  
  const accessFolder = path.basename(req.query.folder || "");  
  const list = fs.readdirSync(path.join(__dirname, 'images', accessFolder));  
  res.json(list);});
```


Proposed changes, however, prevent the browser from displaying the content/images of Cats and Dogs folder.

```
Error: ENOENT: no such file or directory, stat '/home/iana/workshop3/images/cat1.webp'
```

So the code has to change its entire structure. The explanation to exact modifications are provided within the comments in the screenshot below (only the actual changes are pictured, the rest of the code stays intact)

```
import { fileURLToPath } from url;

const __dirname = path.dirname(fileURLToPath(import.meta.url)); // converts url to a filesystem path

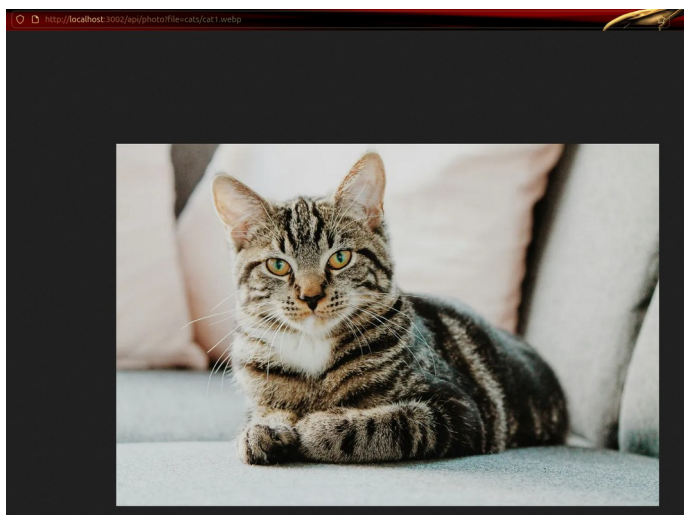
const port = 3002;
const app = express();
app.use(express.static('frontend'));

// build function to join basepath with user function
// prevent traversal by making sure that final path stays within the base folder
function safeJoin(base, userPath) {
  const resolvedPath = path.join(base, userPath || ''); // merge base and user input
  if (!resolvedPath.startsWith(base)) { // if outside of the base...
    throw new Error('Invalid path'); //...throw an error msg
  }
  return resolvedPath; // return to mainpath if confirmed safe
}

app.get('/api/photolist', (req, res) => {
  try {
    const folderPath = safeJoin(path.join(__dirname, 'images'), req.query.folder); //resolve requested folder inside /images
    const list = fs.readdirSync(folderPath); //read dir content
    res.json(list); // return as json
  } catch (err) {
    res.status(400).json({ error: 'Invalid folder path' }); //return 400 error if outside of the defined scope
  }
});

app.get('/api/photo', (req, res) => {
  try {
    const filePath = safeJoin(path.join(__dirname, 'images'), req.query.file); //resolve requested file inside /images
    res.sendFile(filePath); //send filecontent to browser
  } catch (err) {
    res.status(400).send('Invalid file path'); // return 400 error if outside of the scope
  }
});
```

This way the content of the folder is displayed by the browser but any traversal attempts are blocked, yielding the error as per code. This constitutes a successful solution to the traversal attack.



Risk assessment

The traversal vulnerability presented in this workshop is rated as medium to high. The exercises represent traversal vulnerability in both API endpoints `/api/photolist` and `/api/photo`. User-controlled input (`req.query.folder` and `req.query.file`) is directly concatenated into filesystem paths using `path.join()` without validation or sanitization. This allows hackers to use traversal sequences such as `../` to access files outside the intended `/images` directory and way into the system. This shows loss of confidentiality, since internal sensitive files unintended for public view can be read by a client through simple HTTP requests.

The high ranking is due to the fact that the attack is easy to conduct through normal browser tools and network monitoring. The APIs are exposed in plain sight and don't require authentication. The hacking only requires basic knowledge of HTTP requests and filesystem traversal sequences (`../`). No need for advanced tools or fancy hacking techniques.

The impact depends on what files are accessible to the server process. In this case it was possible to obtain and download any sensitive files of choice as root files, system config, `/etc/passwd`, user account and credentials. Getting to critical files leads to further access compromise.

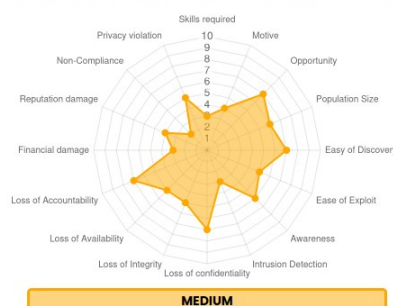
With write privileges there may lie a possibility to open terminal, execute code or modify files (not in the scope of this workshop but as a matter of speculation).

From a defense perspective, several missing protections contribute to the risk:

- No input validation or path normalization checks
- No restriction ensuring paths stay inside the `/images` directory
- No authentication or access control on the API
- No logging or monitoring to detect abnormal requests

Measures would typically include validating user input, rejecting traversal sequences, and verifying that resolved paths remain within the intended directory.

OWASP Risk Assessment Calculator



Threat Agent Factors		Technical Impact Factors	
Skill level	Some technical skills (3)	Loss of confidentiality	extensive critical data disclosed (7)
Motive	Possible reward (4)	Loss of integrity	Extensive slightly corrupt data (5)
Opportunity	Some access or resources required (7)	Loss of availability	Minimal primary services interrupted (5)
Size	Authenticated users (6)	Loss of accountability	Possibly traceable (7)
Vulnerability Factors		Business Impact Factors	
Ease of discovery	Easy (7)	Financial damage	Minor effect on annual profit (3)
Ease of exploit	Easy (5)	Reputation damage	Loss of major accounts (4)
Awareness	Obvious (6)	Non-compliance	Minor violation (2)
Intrusion detection	Logged and reviewed (3)	Privacy violation	Hundreds of people (5)
Likelihood score		Impact score	
5.125 MEDIUM		4.750 MEDIUM	